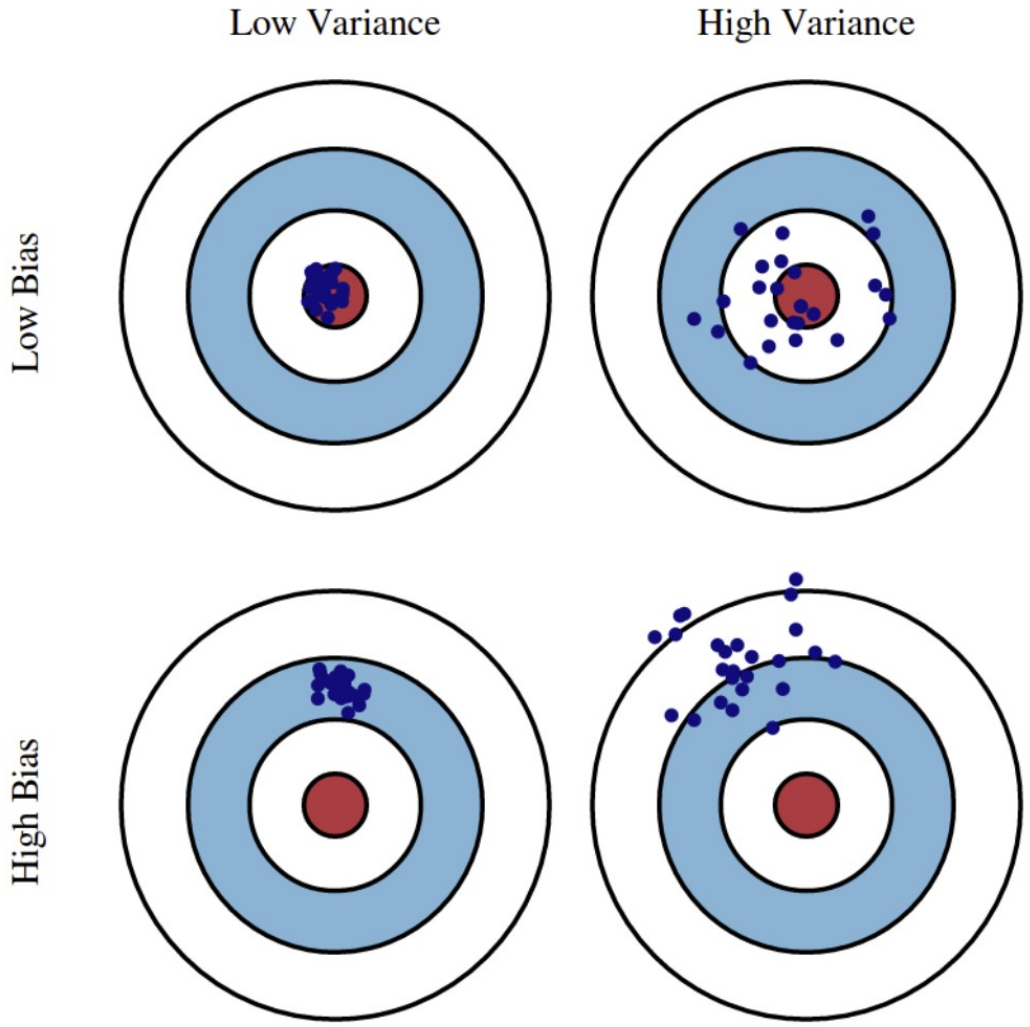
Lecture 6: Regression & Overfitting

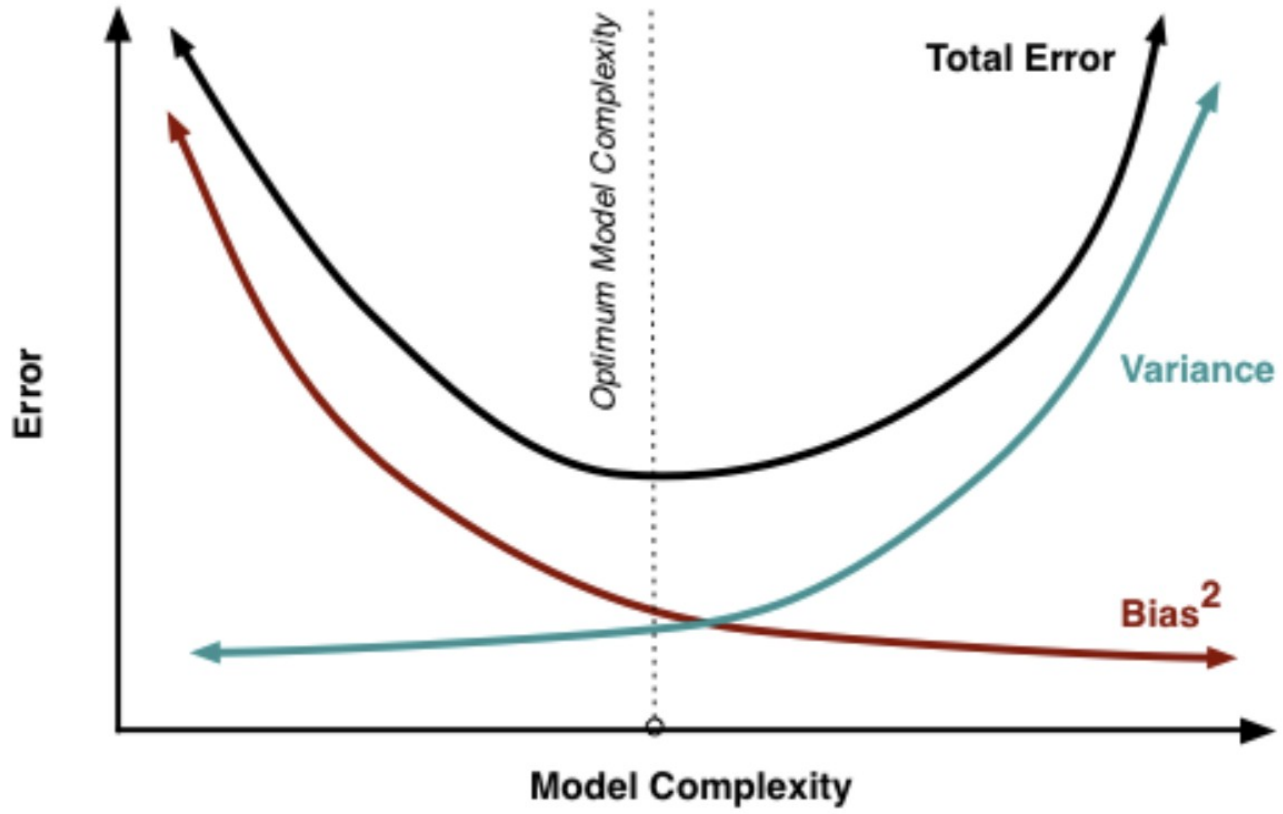
Bias-variance tradeoff

- **Variance**: Captures how much your classifier changes if you train on a different training set. How "over-specialized" is your classifier to a particular training set (“overfitting”)? If we have the best possible model for *our* training data, how far off are we from the average classifier?

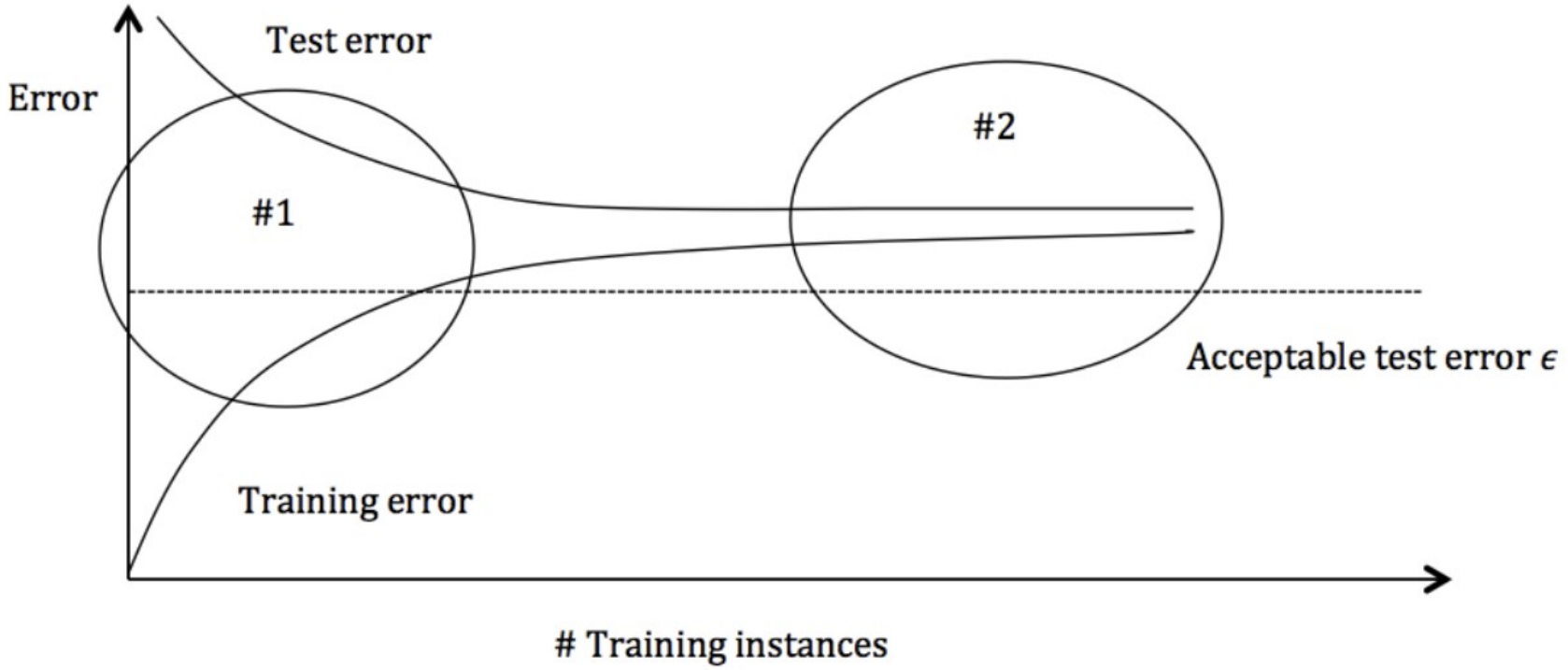
- **Bias**: What is the inherent error that you obtain from your classifier even with infinite training data? This is due to your classifier being "biased" to a particular kind of solution (e.g. linear classifier). In other words, bias is inherent to your model.

Bias-variance *tradeoff*



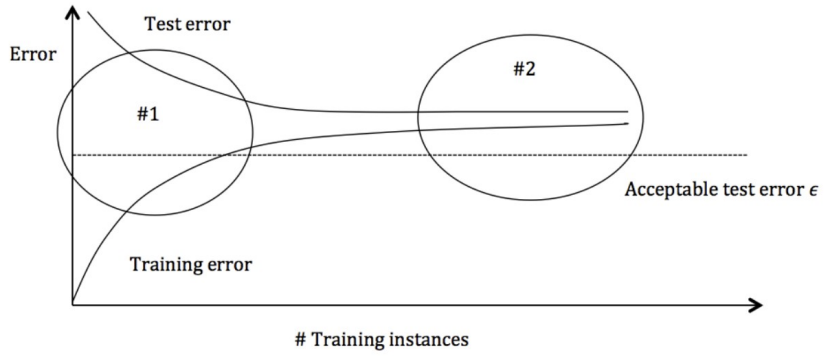


Bias-variance *tradeoff*



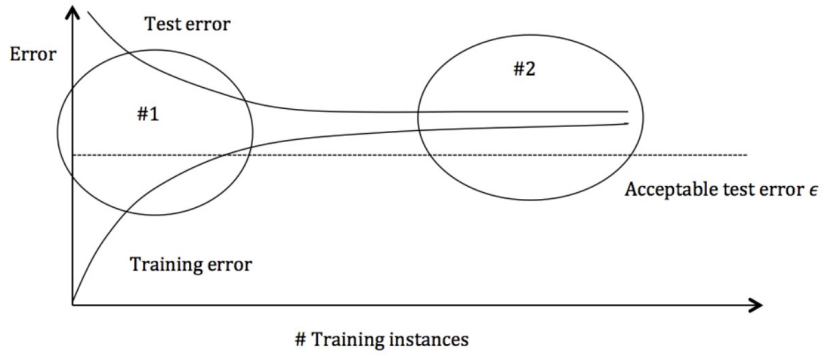
Bias-variance *tradeoff*

- Scenario/plan/regime #1 (high variance):
  - **Variance**: Captures how much your classifier changes if you train on a different training set. How "over-specialized" is your classifier to a particular training set (“overfitting”)? If we have the best possible model for *our* training data, how far off are we from the average classifier?



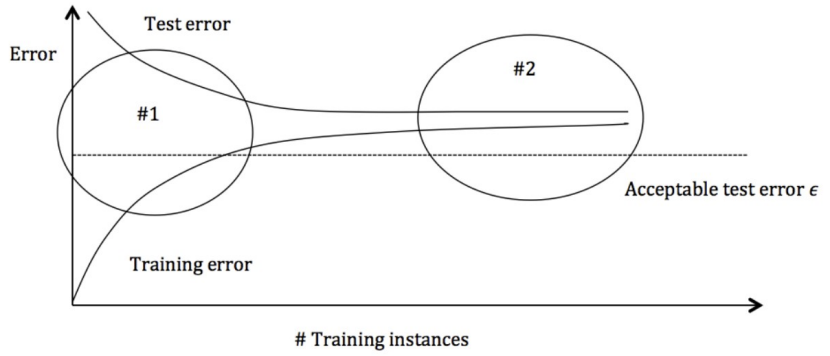
Bias-variance *tradeoff*

- Scenario/plan/regime #1 (high variance):
  - Symptoms:
    - Training error is much lower than test error
    - Training error is lower than ε
    - Test error is greater than ε
  - Remedies:
    - Add more training data
    - Reduce model complexity
    - Bagging



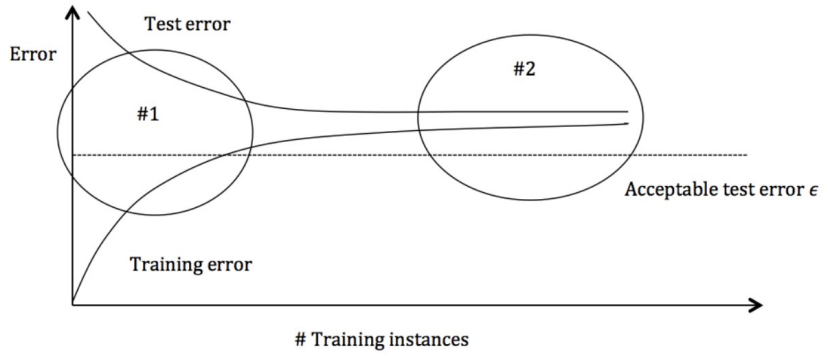
Bias-variance *tradeoff*

- Scenario/plan/regime #2 (high bias):
  - **Bias**: What is the inherent error that you obtain from your classifier even with infinite training data? This is due to your classifier being "biased" to a particular kind of solution (e.g. linear classifier). In other words, bias is inherent to your model.



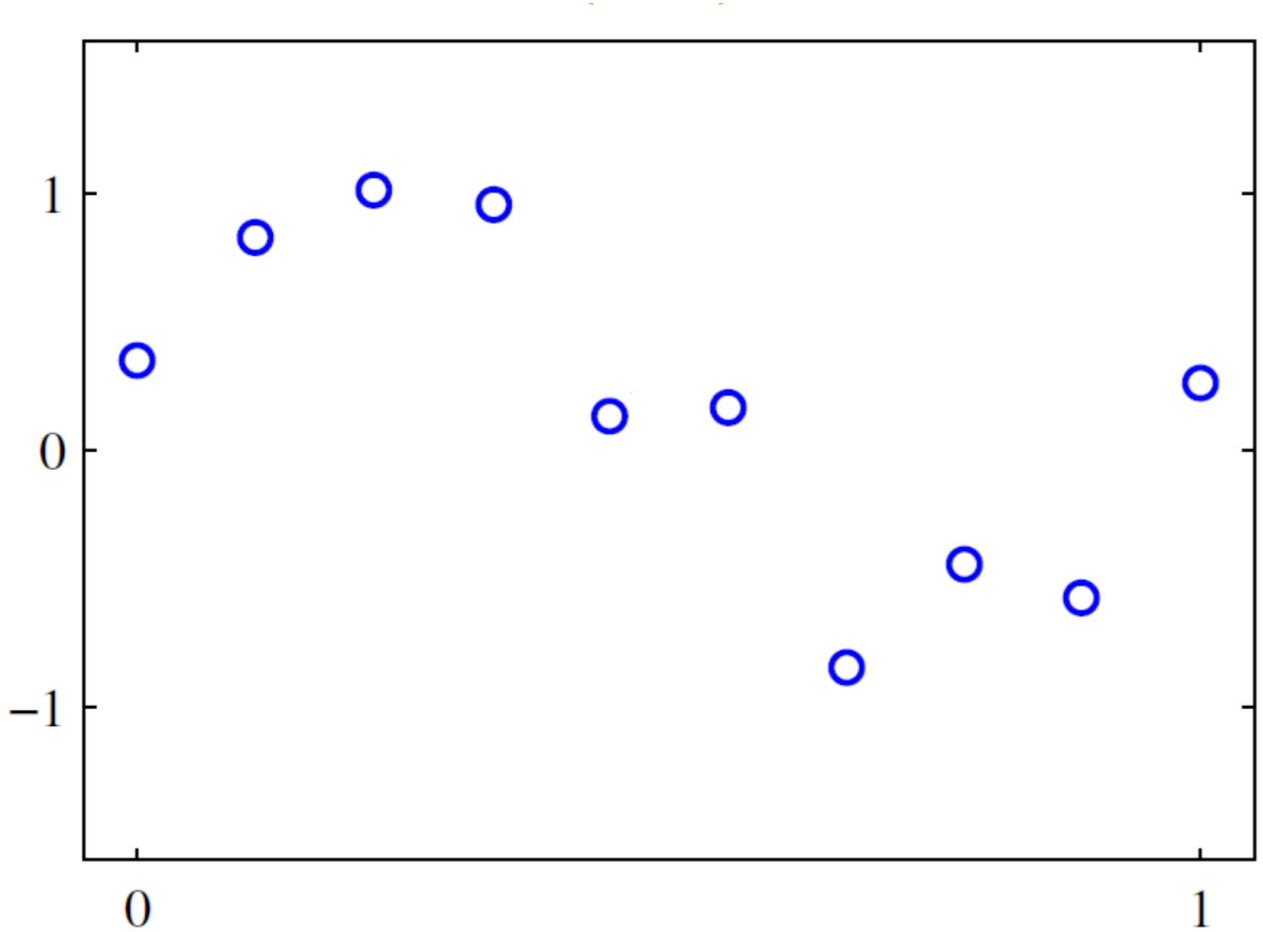
Bias-variance *tradeoff*

- Scenario/plan/regime #2 (high bias):
  - Symptoms (well, symptom):
    - Training error is higher than ε
  - Remedies:
    - Use a more complex model (say, *non*-linear regression)
    - Add features
    - Boosting



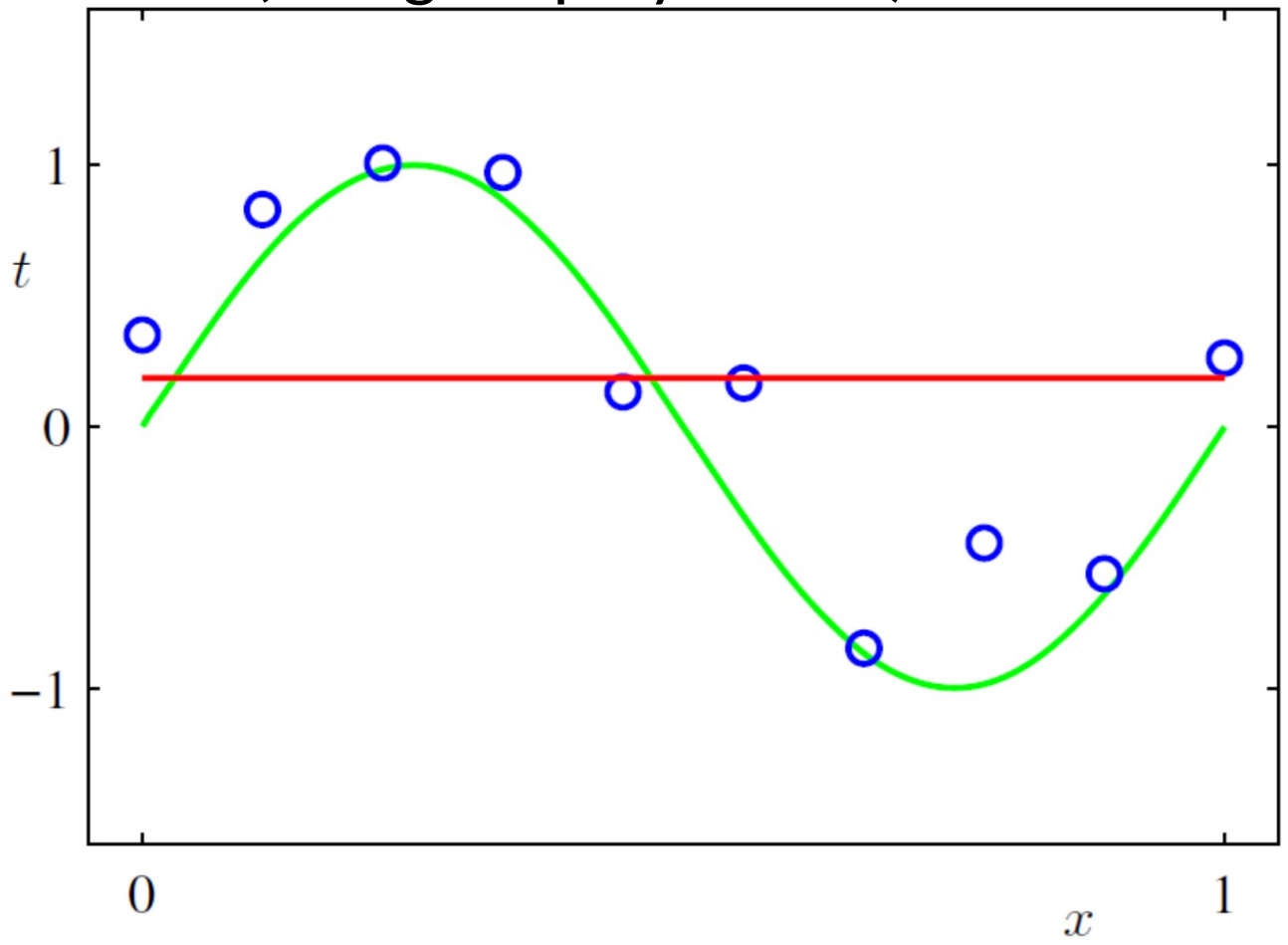
Overfitting

Say you have the following data…



Overfitting

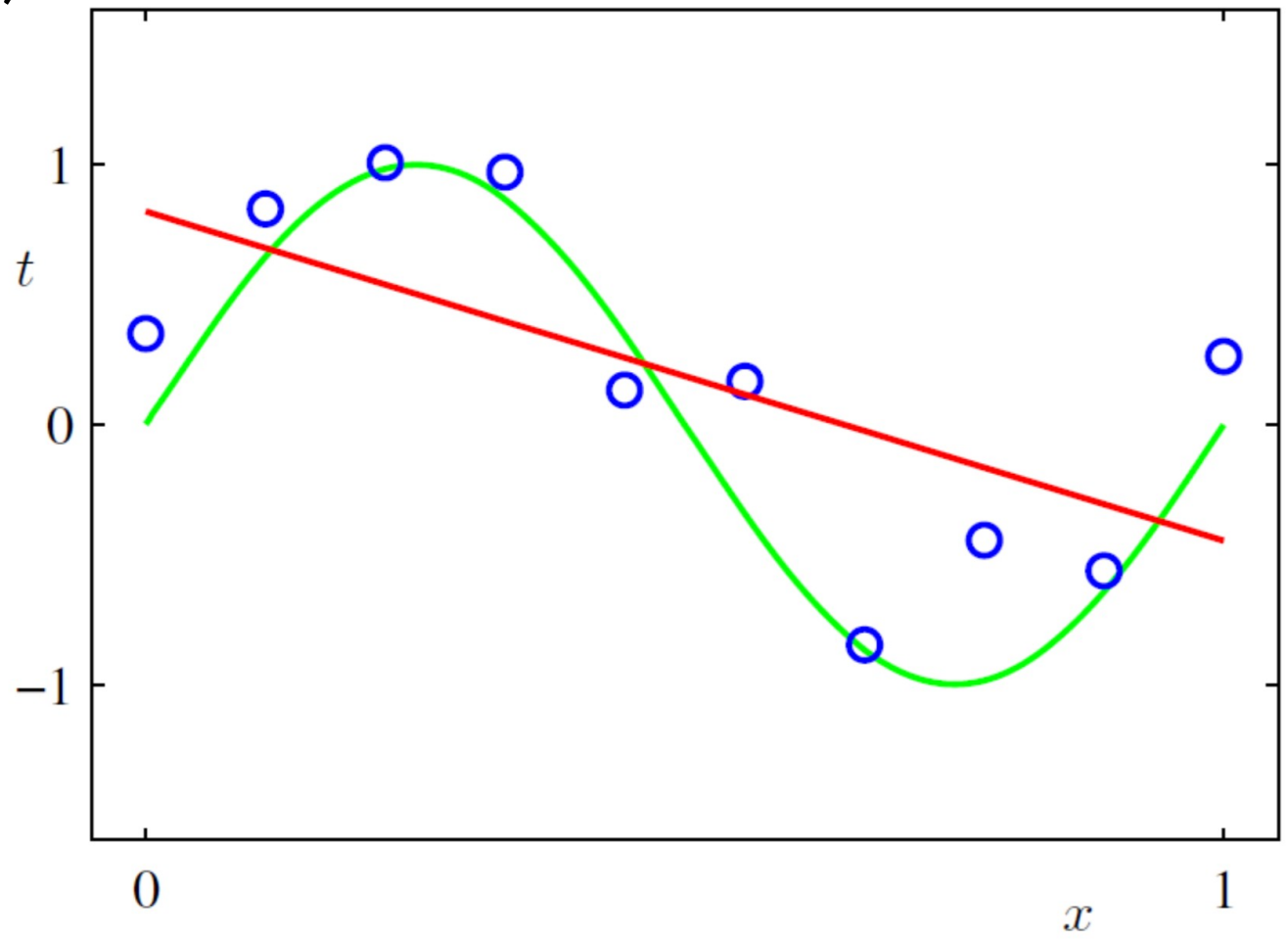
You could fit it like this (0 degree polynomial)…



Overfitting

Or like this (1st)…

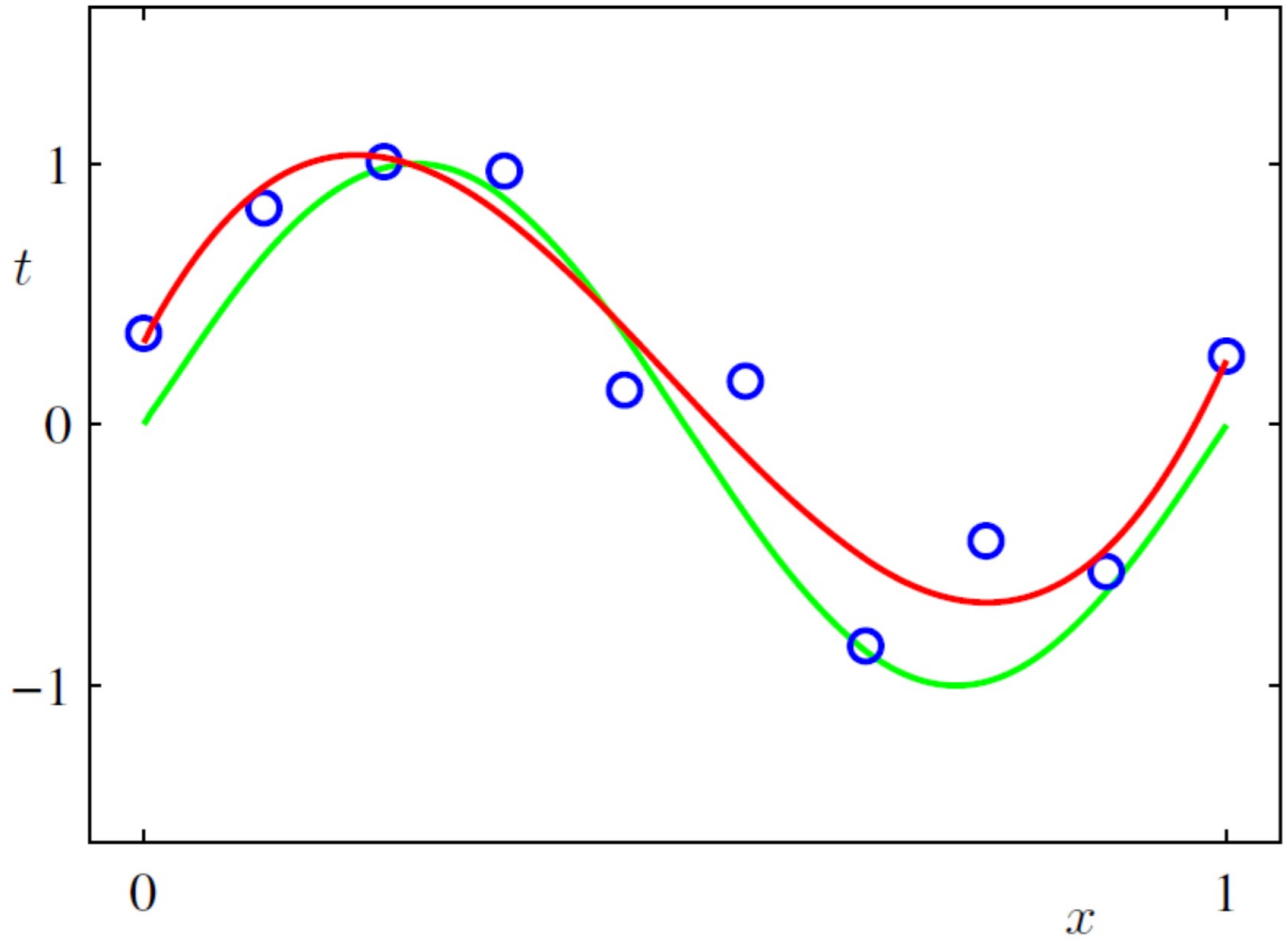
$$t = \sin(2\pi x) + \epsilon$$



Overfitting

Or like this (3rd)…

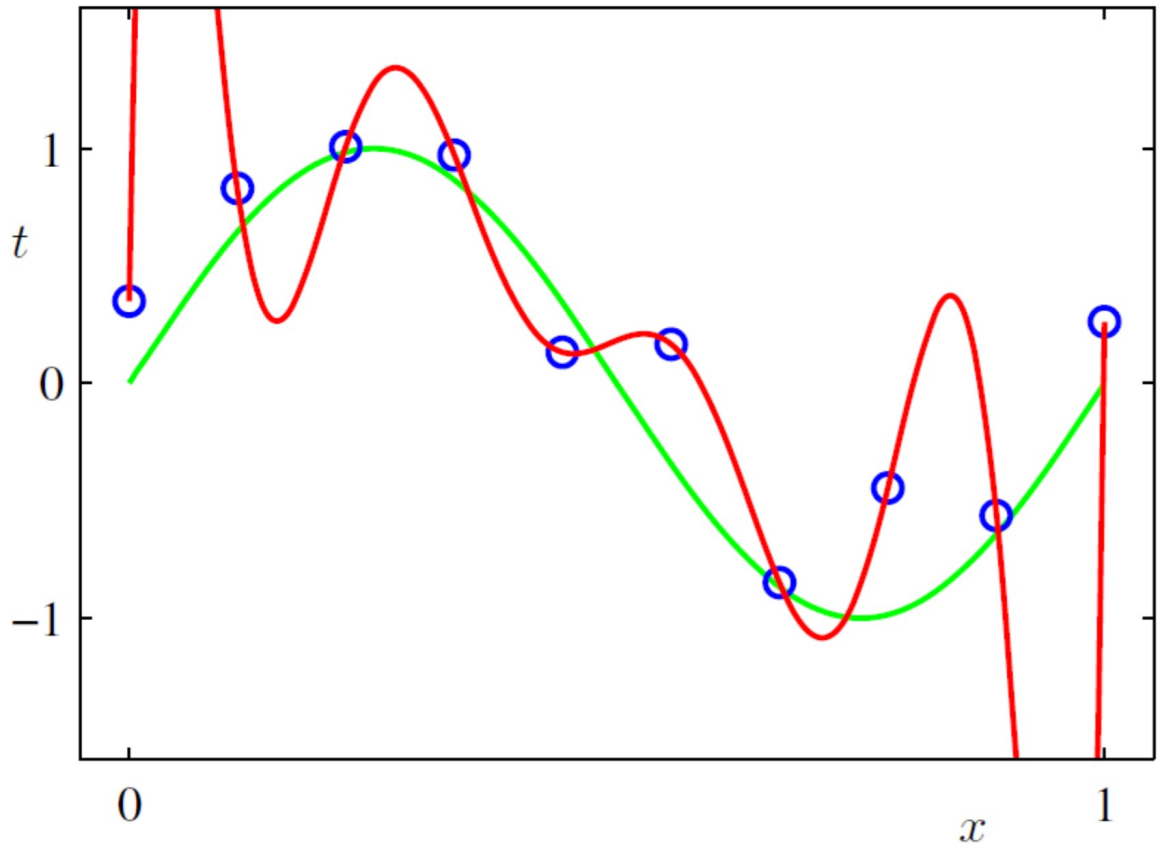
$$t = \sin(2\pi x) + \epsilon$$



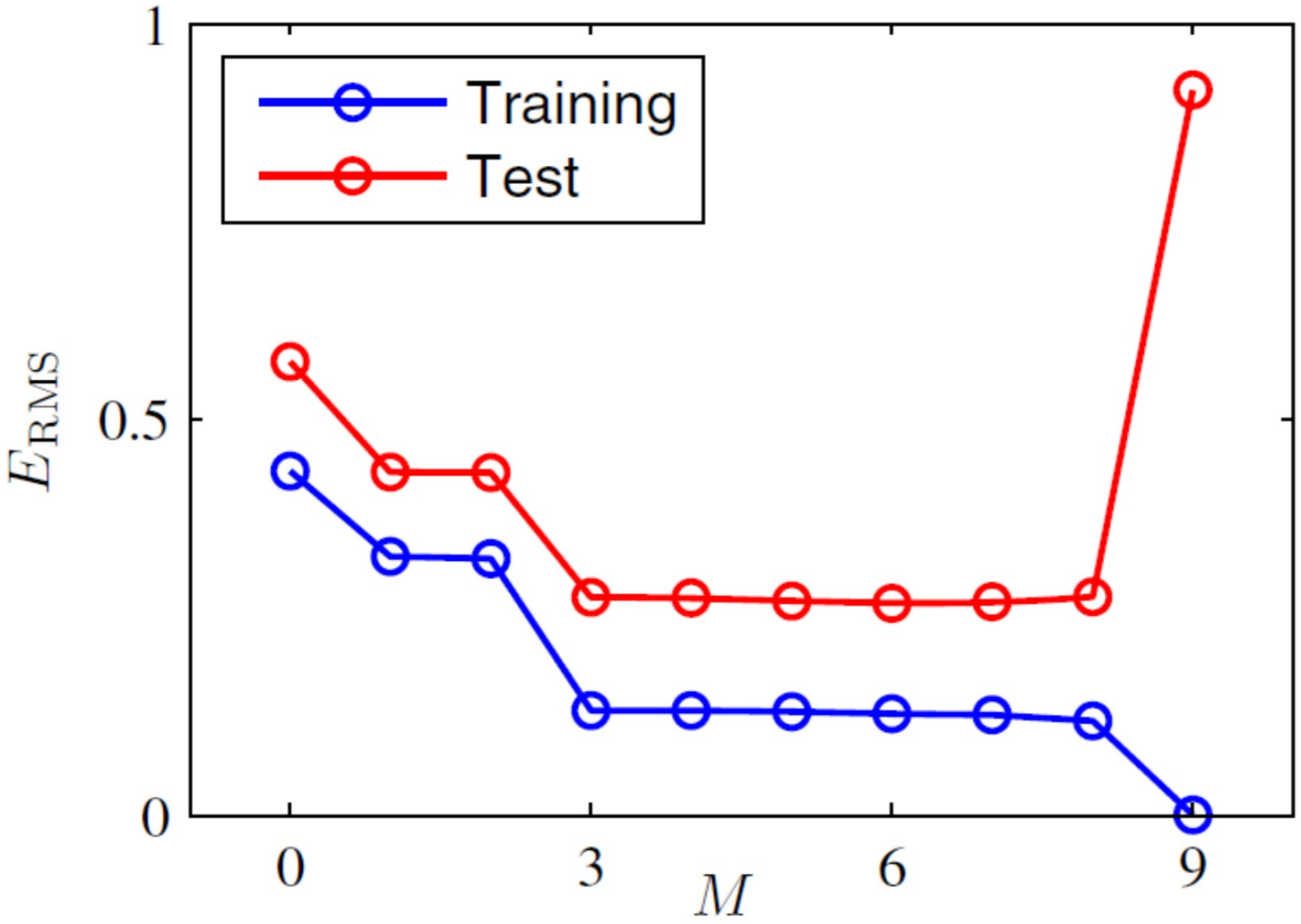
Overfitting

Or like this… (9th degree polynomial)

$$t = \sin(2\pi x) + \epsilon$$



Error vs degree of polynomial



Recap

- So, if you “fit” your training data too well (if you learn too much), you end up describing your training data *very* well, but at the cost of not describing your test data at all (“overfitting”)
- The larger the training data, the less difference there tends to be between the training data and the test data (larger data set helps!)
- The larger the hypothesis class (the more potential models that you allow), the easier it is to find a model that fits very well (and therefore predisposes you more to overfitting)

Too many possible models (hypotheses)?

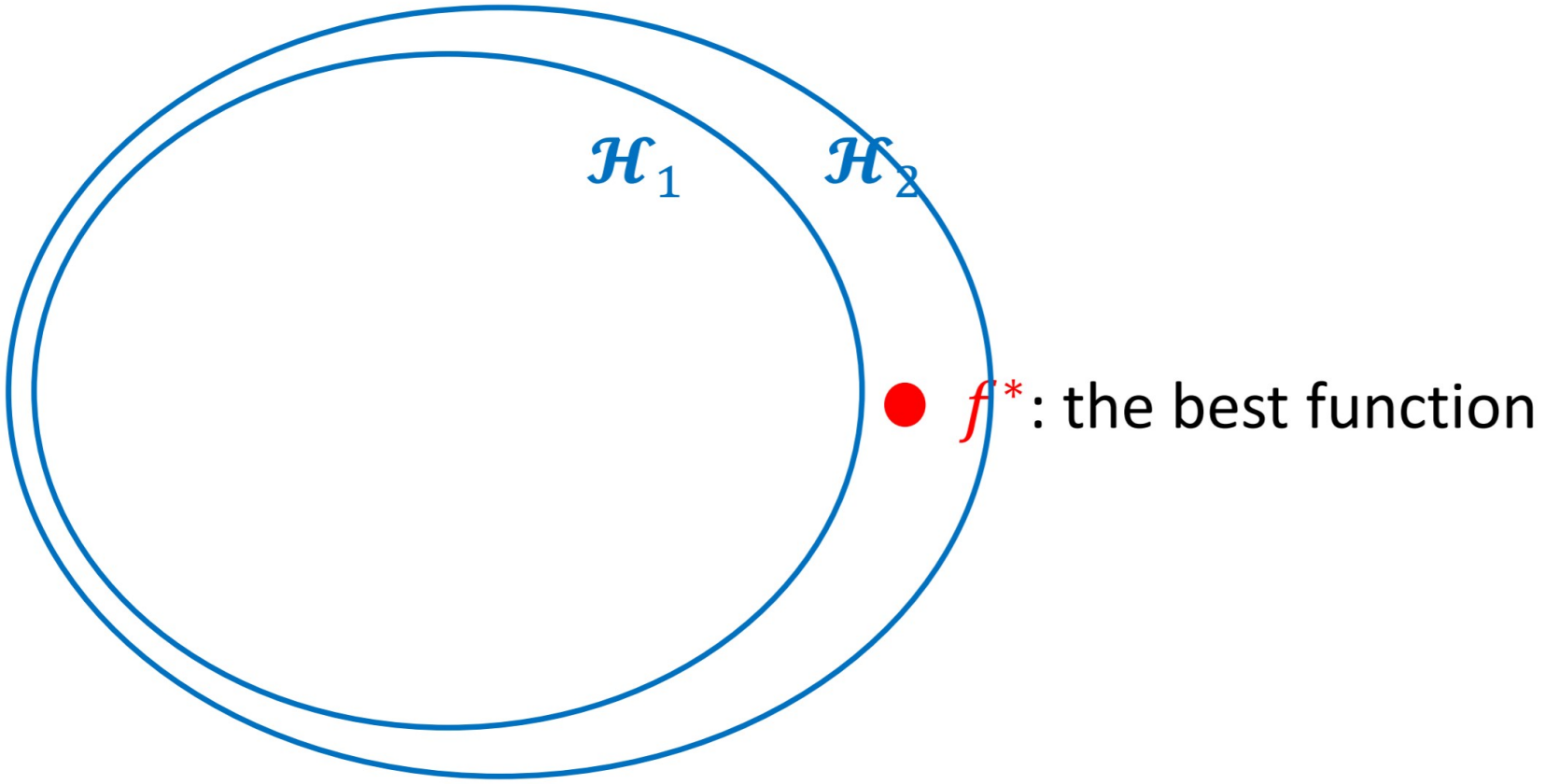
- The larger the hypothesis class (the more potential models that you allow), the easier it is to find a model that fits very well (and therefore predisposes you more to overfitting)
- Therefore, it often is not best to just learn from the data as we did with MLE, but to incorporate prior knowledge, as we did with Bayesian inference
  - But this can be overcome with more data! And is a *good strategy* **if you have no reliable prior knowledge**
  - But throwing out useless hypotheses is always a good idea

Prior knowledge vs data?

- Let's start by looking at the extremes:
  - "Perfect" prior knowledge – you don't need data, your model is equal to the true underlying distribution
  - "Perfect" data – you have infinite data, so your data *is* the underlying distribution

- Practical scenarios: finite data, H, actual model function f^* (you don't know this)

f* in or not in H



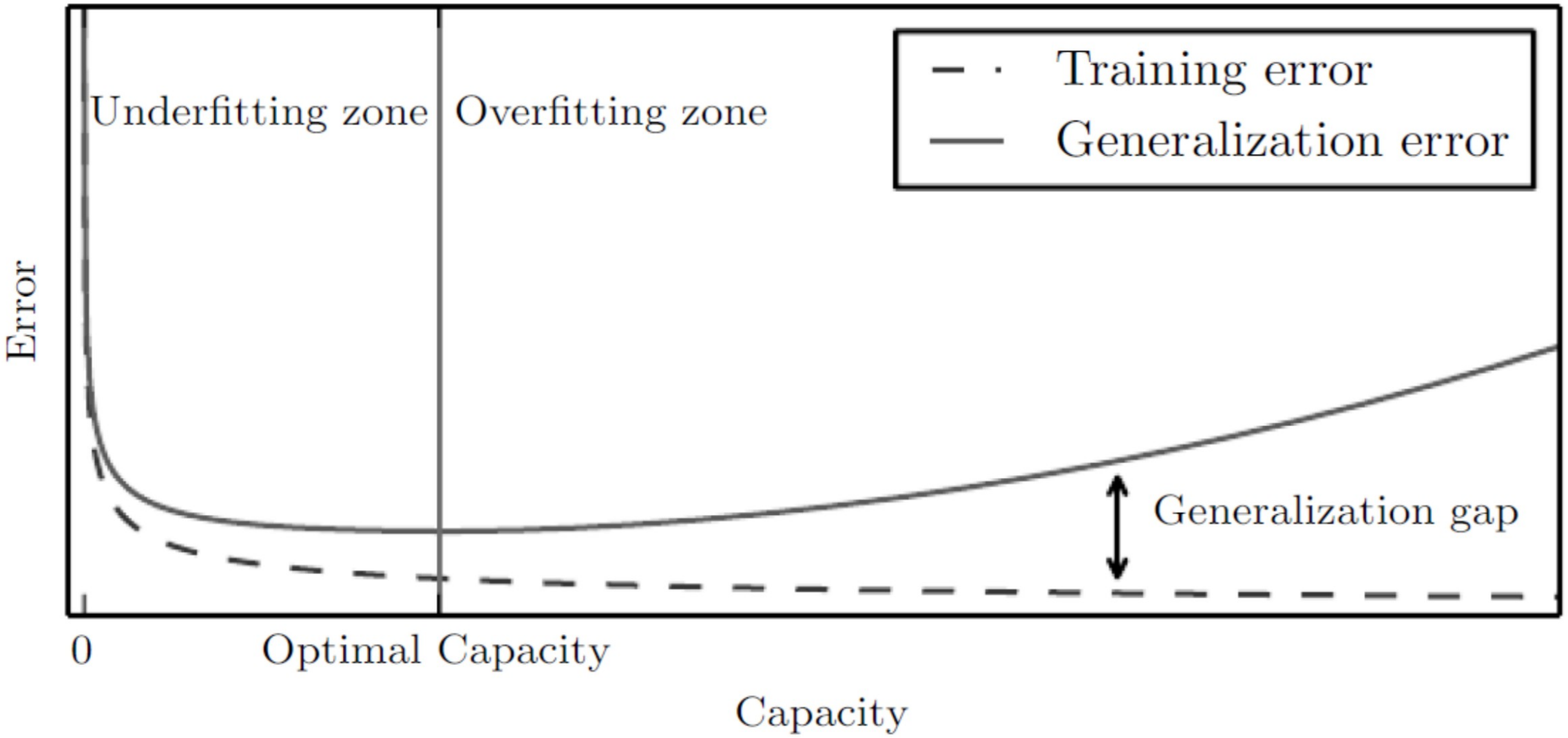
As you get more data, H_1 becomes H_2 and is more likely to encompass the actual model f^*

Prior knowledge vs data?

- Practical scenarios lie between the two extreme cases



Prior knowledge vs data?



Optimal capacity

- How do we select optimal capacity, for example, the best degree for polynomial curve fitting?
- Cannot be done by training data alone (it will always look better to just overfit)
- Solution is simple and easy to understand – cross validation

Cross validation

- Partition your training data into several groups (number of groups varies, but usually at least 3)
- Run your machine learning algorithm using all of the training data except for one of these groups
- Then use the group that you left out as the “test set”
- Then repeat but with a different group left out, etc., etc.

Cross validation

- We will learn (many) other methods for dealing with how well models are working, but cross validation:
  - Is general (you can almost always partition training data)
  - Is simple
- But…
  - Is computationally expensive, especially when there are a lot of parameters to learn